

# Manipulação *de arquivos III*

Os dois formatos de texto estruturado mais usados para trocar dados: **CSV** (tabelas, planilhas, exportações de sistemas) e **JSON** (configurações, APIs, dados com listas e dicionários aninhados). O Python traz os módulos `csv` e `json` na biblioteca padrão. Nesta aula: gravar e ler CSV com listas e com dicionários, o CSV do Excel brasileiro, gravar e ler JSON em arquivo e em texto, e como tratar tipos que o JSON não conhece.

`csv.writer` · `reader`

`DictReader` · `DictWriter`

`newline=""`

`delimiter=";"`

`json.dump` · `load`

`dumps` · `loads`

`default=`

Formação Claude Code 2026 · Baseado em docs.python.org (csv — CSV File Reading and Writing; json — JSON encoder and decoder; Standard Encodings: utf-8-sig), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

## Uma tabela *em texto*

csv\_escrever.py

```
import csv

linhas = [
    ["produto", "preco", "estoque"],
    ["Caneta", 2.5, 120],
    ["Caderno, 96 fls", 18.9, 40],    # vírgula no texto
]

with open("produtos.csv", "w", encoding="utf-8",
        newline="") as f:
    csv.writer(f).writerows(linhas)

with open("produtos.csv", encoding="utf-8") as f:
    print(f.read(), end="")
```

SAÍDA

```
produto,preco,estoque
Caneta,2.5,120
"Caderno, 96 fls",18.9,40
```

- ✓ Cada lista vira uma linha; os itens são separados por vírgula
- ✓ O `csv` põe aspas sozinho quando o texto tem vírgula ("Caderno, 96 fls")
- ✓ Números viram texto: o arquivo é só texto

Abra arquivos CSV sempre com `newline=""`. Sem isso, aparecem linhas em branco extras no Windows.

## csv.reader: *uma lista por linha*

csv\_ler.py

```
import csv

with open("produtos.csv", encoding="utf-8", newline="") as f:
    leitor = csv.reader(f)
    cabecalho = next(leitor)           # primeira linha
    for linha in leitor:
        print(linha)

print(cabecalho)
print(type(linha[1]))                # tudo chega como texto
```

SAÍDA

```
['Caneta', '2.5', '120']
['Caderno, 96 fls', '18.9', '40']
['produto', 'preco', 'estoque']
<class 'str'>
```

- ✓ `csv.reader` é um iterador: entrega uma lista de textos por linha
- ✓ `next(leitor)` tira o cabeçalho antes do `for` (aula 01)
- ✓ As aspas somem e a vírgula de dentro do texto fica no lugar certo
- ✓ Tudo chega como `str`: converta com `int()` e `float()`

## DictReader e DictWriter

csv\_dict.py

```
import csv

with open("produtos.csv", encoding="utf-8", newline="") as f:
    for item in csv.DictReader(f):
        valor = float(item["preco"]) * int(item["estoque"])
        print(f'{item["produto"]:<16} R$ {valor:>8.2f}')

clientes = [{"nome": "Ana", "cidade": "Recife"},
             {"nome": "Bia", "cidade": "Natal"}]
with open("clientes.csv", "w", encoding="utf-8",
          newline="") as f:
    escritor = csv.DictWriter(f, fieldnames=["nome", "cidade"])
    escritor.writeheader()
    escritor.writerows(clientes)
print(open("clientes.csv", encoding="utf-8").read(), end="")
```

SAÍDA

```
Caneta          R$   300.00
Caderno, 96 fls R$   756.00
nome,cidade
Ana,Recife
Bia,Natal
```

- ✓ `DictReader` usa o cabeçalho como chaves: `item["preco"]` em vez de `linha[1]`
- ✓ O código continua certo mesmo se a ordem das colunas mudar
- ✓ `DictWriter` faz o caminho inverso; `writeheader()` grava o cabeçalho

Para planilhas com cabeçalho, prefira as versões com dicionário: o código fica autoexplicativo.

## Ponto e vírgula *e acentos no Excel*

csv\_excel.py

```
import csv

dados = [{"mês", "receita"}, {"outubro", "1.250,90"}]

# O Excel em português costuma usar ; como separador
with open("excel_br.csv", "w", encoding="utf-8-sig",
          newline="") as f:
    csv.writer(f, delimiter=";").writerows(dados)

with open("excel_br.csv", encoding="utf-8-sig",
          newline="") as f:
    for linha in csv.reader(f, delimiter=";"):
        print(linha)
```

SAÍDA

```
['mês', 'receita']
['outubro', '1.250,90']
```

- ✓ Com o Excel configurado em português, a vírgula é o separador decimal; por isso o separador de colunas costuma ser `;`
- ✓ `delimiter=";"` vale tanto para gravar quanto para ler
- ✓ `utf-8-sig` grava uma marca no começo que ajuda o Excel a reconhecer os acentos

"1.250,90" continua texto: para fazer conta, troque `.` por nada e `,` por `.` antes do `float()`.

## json.dump e json.load

json\_basico.py

```
import json

config = {"tema": "escuro", "fonte": 14, "atalhos": ["ctrl+s"],
          "beta": True, "proxy": None}

with open("config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2, ensure_ascii=False)

with open("config.json", encoding="utf-8") as f:
    lido = json.load(f)

print(lido == config, type(lido["atalhos"]).__name__)
print(open("config.json", encoding="utf-8").read())
```

SAÍDA

```
True list
{
  "tema": "escuro",
  "fonte": 14,
  "atalhos": [
    "ctrl+s"
  ],
  "beta": true,
  "proxy": null
}
```

| PYTHON                                 | JSON                                   |
|----------------------------------------|----------------------------------------|
| <code>dict</code>                      | objeto <code>{}</code>                 |
| <code>list</code> , <code>tuple</code> | array <code>[]</code>                  |
| <code>True</code> / <code>False</code> | <code>true</code> / <code>false</code> |
| <code>None</code>                      | <code>null</code>                      |

- ✓ `indent=2` deixa o arquivo legível para pessoas
- ✓ Ao ler, volta tudo como `dict`, `list`, `str`, números e `None`

## .dumps e loads: JSON em string

json\_texto.py

```
import json

pedido = {"cliente": "João", "itens": [{"sku": "A1", "qtd": 2}]}

texto = json.dumps(pedido, ensure_ascii=False)  # vira string
print(texto)
print(json.dumps(pedido))                      # sem ensure_ascii

volta = json.loads(texto)                      # string -> dict
print(volta["itens"][0]["qtd"])
```

SAÍDA

```
{"cliente": "João", "itens": [{"sku": "A1", "qtd": 2}]}
{"cliente": "Jo\u00e3o", "itens": [{"sku": "A1", "qtd": 2}]}
2
```

- ✓ Com **s** no fim ( **dumps** , **loads** ), o JSON vai para ou vem de uma **string**
- ✓ É o que se envia e recebe de APIs na web
- ✓ Sem **ensure\_ascii=False** , acentos viram códigos como **ã** (continua válido, mas ilegível)

Regra para lembrar: sem **s** , arquivo; com **s** , string.

## Datas e sets: *o que o JSON não aceita*

json\_tipos.py

```
import json
from datetime import date

evento = {"nome": "Workshop", "data": date(2026, 11, 20),
          "tags": {"python", "dados"}}

try:
    json.dumps(evento)
except TypeError as erro:
    print("Erro:", erro)

def converter(valor):
    if isinstance(valor, date):
        return valor.isoformat()
    if isinstance(valor, set):
        return sorted(valor)
    raise TypeError(type(valor))

print(json.dumps(evento, default=converter, ensure_ascii=False))
```

SAÍDA

```
Erro: Object of type date is not JSON serializable
{"nome": "Workshop", "data": "2026-11-20", "tags": ["dados",
"python"]}
```

- ✓ O JSON só conhece texto, número, booleano, nulo, lista e objeto
- ✓ `date`, `set` e objetos próprios dão `TypeError`
- ✓ `default=função` é chamado para cada valor desconhecido e devolve algo que o JSON aceita
- ✓ Datas no formato ISO ( `2026-11-20` ) são o padrão de mercado

## De CSV *para* JSON

**Enunciado:** leia um CSV de vendas, some o total de cada vendedor e grave o resultado num arquivo JSON.

Uma resposta possível

```
import csv
import json
from collections import defaultdict

with open("vendas.csv", "w", encoding="utf-8", newline="") as f:
    f.write("vendedor,valor\nAna,300\nBia,120\n")
    f.write("Ana,80\nCaio,50\n")

totais = defaultdict(float)
with open("vendas.csv", encoding="utf-8", newline="") as f:
    for venda in csv.DictReader(f):
        totais[venda["vendedor"]] += float(venda["valor"])

with open("totais.json", "w", encoding="utf-8") as f:
    json.dump(totais, f, indent=2, ensure_ascii=False)

print(open("totais.json", encoding="utf-8").read())
```

SAÍDA

```
{
  "Ana": 380.0,
  "Bia": 120.0,
  "Caio": 50.0
}
```

- ✓ `DictReader` lê, `defaultdict` soma (aula 05) e `json.dump` grava
- ✓ Um `defaultdict` é um `dict`: o JSON aceita direto
- ✓ Desafio: grave também a quantidade de vendas de cada um

## Teste o que *ficou*

1. Ao ler um CSV com `csv.reader`, os números chegam como:

A) int e float

B) str

C) bytes

D) None

B.

2. Para acessar as colunas pelo nome do cabeçalho:

A) `csv.reader`

B) `csv.DictReader`

C) `json.load`

D) `open(..., "r")`

B.

3. `json.dumps` devolve:

A) Um dicionário

B) Uma string

C) Um arquivo

D) Bytes

B.

4. `None` do Python vira, em JSON:

A) `"None"`

B) `null`

C) `0`

D) `false`

B.

# Tabelas e objetos *em texto*

CSV com `newline=""`: `writer` e `reader` trabalham com listas, `DictWriter` e `DictReader` com o cabeçalho, e `delimiter=";"` com `utf-8-sig` atendem o Excel brasileiro. JSON: `dump` / `load` para arquivos, `dumps` / `loads` para strings, `indent` e `ensure_ascii=False` para legibilidade e `default=` para tipos desconhecidos.

csv

DictReader

delimiter=";"

json.dump · load

default=

Próxima aula → 19 · Manipulação de Arquivos IV

Fontes: [docs.python.org/3/library/csv.html](https://docs.python.org/3/library/csv.html); [docs.python.org/3/library/json.html](https://docs.python.org/3/library/json.html); [docs.python.org/3/library/codecs.html#standard-encodings](https://docs.python.org/3/library/codecs.html#standard-encodings) (`utf-8-sig`); [docs.python.org/3/library/datetime.html#datetime.date.isoformat](https://docs.python.org/3/library/datetime.html#datetime.date.isoformat), consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.